CS236 Spring 2024

Lab Quiz 2

5 questions, 25 marks

Instructions

Before you begin, please check that you can see the following files on the Desktop:

- This question paper `labquiz2.pdf`
- The original unmodified xv6 tarball. You can untar this file to obtain the xv6 code folder:
  `tar -zxvf xv6-public.tgz`
- A tarball `labquiz2_code.tgz` You can untar this file to obtain the required helper code for all the questions:
  `tar -zxvf labquiz2_code.tgz`
  This folder contains the code for each question in a separate subdirectory, with names `q1`, `q2`, `q3`, `q4`, `q5`.

Questions 1–3 are based on xv6. Start with the original unmodified xv6 code for each question. Copy the modified files given to you into this folder. You can then make, build and run the xv6 code as you would do normally. Please remember to start with a fresh unmodified xv6 folder for each question.

Questions 4–5 deal with IPC. You are given template code that you must extend as explained in the questions.

To submit your code, create your submission folder titled `submission_rollnumber` in the same directory, where you must replace the string `rollnumber` above with your own roll number (e.g., your directory name should look like `submission_12345678`). Create separate subdirectories for each question in this folder with names `q1`, `q2`, `q3`, `q4`, `q5`. Place the files you wish to submit for each question in these subdirectories.

For the xv6 questions, you must submit all source code files that you have changed. You must keep track of the files you have changed, and carefully submit all of them. We cannot grade your submission if some files have not been submitted.

For the IPC questions, you must change the template code provided to you and submit those files only. Please do not submit any other files.

Submission Instructions

1. Once you are ready to submit, please run the command `check` from your terminal. This command will create a tarball of your submission folder. If the folder is not in the proper place or is empty, this script will throw an error. In that case, please fix the errors and try again.

2. Next, call one of the TAs and ask them to run the `submit` command from your terminal. The TA will enter the password to upload your submission tarball to our remote submission server. Please note that we will only be grading the files that are submitted in this manner. So please ensure that you submit the correct files.

3. **IMPORTANT NOTE: Please ensure that you are submitting the correct files with the correct filenames. Test your code properly before submission. Strictly NO code changes will be allowed after the exam.**

Help for xv6 code

Listed below are the important xv6 files you will need to understand for solving this quiz.

- `user.h` contains the system call definitions in xv6 that are available to user programs.
- `usys.S` contains a list of system calls exported by the kernel, and the corresponding invocation of the trap instruction.
- `syscall.h` contains a mapping from system call name to system call number. Every system call must have a number assigned here.
- `syscall.c` contains helper functions to parse system call arguments, and pointers to the actual system call implementations.
- `sysproc.c` contains the implementations of process related system calls.
- `defs.h` is a header file with function definitions in the kernel.
- `proc.h` contains the `struct proc` structure.
- `proc.c` contains implementations of various process related system calls, and the scheduler function. This file also contains the definition of `ptable`, so any code that must traverse the process list in xv6 must be written here.

To run xv6 code in QEMU, in the xv6 folder, you must do `make`, followed by `make qemu` or `make-qemu-nox`. You can use Ctrl+A X to exit QEMU.

Exercises

1. **[5 marks]** In xv6, when a parent process P terminates while the child C is still running, the init process becomes the parent of the orphan child C. So, when C eventually terminates and becomes a zombie, it is reaped when init calls wait(). We would like to change this behavior so that an orphan child is adopted by its closest living ancestor, so that the burden to reap orphan processes does not fall on the init process alone. For example, if the grandparent of C, say G, is alive when C becomes a zombie, then C can be reaped when G calls wait(). Your changes must be limited to the implementation of the `exit` system call in `proc.c`.

   In the modified xv6 files given to you, we have implemented the `getppid` system call, which can be used to test your implementation, and ensure that the parent pointer has been correctly updated. We have also provided you with two testcases, which have been added to the Makefile. You can also write your own simple testcases to test your solution. We will grade your solution with new testcases as well.

   In the first testcase given to you, P1 forks P2 and P2 forks P3. P2 exits and is reaped by its parent P1. P3 is re-parented to P1, so that when P3 exits, it is reaped by P1. The expected output when you run this testcase is shown below. The output may be different before you make the required changes. The values of PIDs may also differ slightly in your output.

```
$ q1_tc1
P1: pid = 6
P2 : pid = 7, ppid = 6
P3 : pid = 8, ppid = 7
P2 exiting
P1 reaped process PID 7
P3 : pid = 8, ppid = 6
P1 reaped process PID 8
```

Figure 1: Testcase-1

In the second testcase, the following events happen.

(1) P1 forks P2

(2) P2 forks P3

(3) P3 forks P4 and P5 both. Process tree at this point is given by Figure 2.

(4) P3 dies and is reaped by P2. So, P4 and P5 are re-parented to P2. Process tree at this point is given by Figure 3.

(5) P2 dies and is reaped by P1. So, P4 and P5 are re-parented to P1. Process tree at this point is given by Figure 4.

(6) P4 and P5 die both die and are reaped by P1.

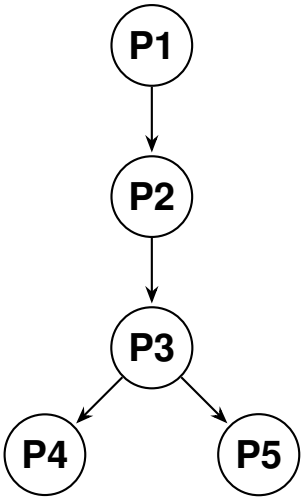

Figure 2: Tree after event (3)

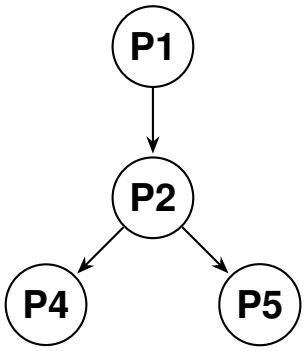

Figure 3: Tree after event (4)

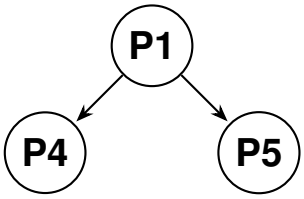

Figure 4: Tree after event (5)

The expected output from the second testcase, when your solution runs correctly, is shown below. The output may be different before you make the required changes. The values of PIDs may also differ slightly in your output.

```
$ q1_tc2
P1: pid = 3
P2 : pid = 4, ppid = 3
P3 : pid = 5, ppid = 4
P4 : pid = 6, ppid = 5
P5 : pid = 7, ppid = 5
P3 exiting
P2 reaped process PID 5
P4 : pid = 6, ppid = 4
P5 : pid = 7, ppid = 4
P2 exiting
P1 reaped process PID 4
P4 : pid = 6, ppid = 3
P4 exiting
P1 reaped process PID 6
P5 : pid = 7, ppid = 3
P5 exiting
P1 reaped process PID 7
```

Figure 5: Testcase-2

2. **[5 marks]** Implement the functionality to trace system calls in xv6. We have added a new system call `strace` in the modified files, which currently does not do anything. Change the code of this system call in such a way that, after a process invokes this system call, all future system calls made by the process should generate a debug output to the screen of the format `system call invoked: <NUMBER> <NAME> by PID <PID>` where NUMBER is the system call number and NAME is the name of the system call (e.g., `fork`). This debug output should be printed to screen until the process finishes. We have given the string names of the system calls that you must print in `syscall_strings.h` for your convenience.

   Note that this tracing functionality should NOT be passed to children during fork, and should be reset on making an exec system call. You will likely add some field to the `struct proc` to track this tracing functionality, and you must reset this field during fork and exec.

   We have given you two test cases with different levels of difficulty to test your code. These testcases have been added to the Makefile as well. Once your solution is complete, the expected output from the testcases should look like what is shown below.

```
$ testcase1

system call invoked: 13 sleep by PID 3

system call invoked: 13 sleep by PID 3

system call invoked: 16 write by PID 3
hello

system call invoked: 2 exit by PID 3
```

Figure 6: Testcase-1

```
$ testcase2

system call invoked: 16 write by PID 5
o
system call invoked: 16 write by PID 5
u
system call invoked: 16 write by PID 5
t
system call invoked: 16 write by PID 5


system call invoked: 2 exit by PID 5
out
```

Figure 7: Testcase-2

3. **[5 marks]** Write a system call `get_sibling_info()` to print information about the siblings of a process to the screen. The system call does not take any arguments and does not return any value. When invoked, it prints information about the siblings (i.e., other processes with the same parent process). Every invocation of the system call should print a header line saying what fields will be printed, followed by one line of output for each sibling, with the following information: the PID of the sibling, the state of the sibling process (e.g., running, runnable, sleeping, ..) in a string format, and the command name invoked to run the process. The command name is simply the command given to a process during exec. You might want to look at the code of the `exec` syscall to know how the command name is set. Note that a child inherits the parent's command name during fork.

   We have given you one big testcase and the expected output, as shown below. Feel free to write your own simpler testcases as well. Note that we changed the number of CPUs to 1 in the `Makefile` to make the output deterministic.

```
$ testcase1
PID     STATE   COMMAND
4       SLEEPING        testcase1
PID     STATE   COMMAND
4       SLEEPING        testcase1
5       SLEEPING        testcase1
PID     STATE   COMMAND
5       SLEEPING        testcase1
$
```

Figure 8: Testcase-1

4. **[3 marks]** In this question, you are given two simple programs for a pipe reader and writer. Both programs open a named pipe. The writer program given to you sends the string "hello" to the reader program through the pipe. The reader program reads the string from the pipe and prints it to screen. You must modify these programs so that the writer program continuously prompts the user to enter a message (up to 64 bytes), and sends this message to the pipe reader, which then displays it to the screen. The writer and reader programs must run in an infinite loop until terminated by Ctrl+C. Do not print anything except the received message on the reader side. Do not put any extra `endl` or `\n`.

   The template code given to you can be compiled and run as shown below.

   ```
   gcc -o writer writer.c && ./writer
   ```

   ```
   gcc -o reader reader.c && ./reader
   ```

   You can change the template code itself for your solution, and submit it. Please test your code thoroughly by sending different types of messages.

5. **[7 marks]** In this question, you will implement a mechanism to transfer the data in a file from one process to the other via connection-less Unix domain sockets. You are given a template sender program that takes a filename as commandline argument, reads the file from disk, and displays its contents to screen. You must change this program to send the file data via sockets to the receiver program. You must also change the code for the receiver program to receive the file from the sender via the socket and display its contents to the screen. The template code already contains code for opening sockets and reading the file. You must add code to send and receive from the sockets, e.g., using `sendto` and `recvfrom` system calls.

   Ideally, we want the sender and receiver to terminate once the file transfer completes. Some part of the grade (2 marks) will only be given to students who make the sender and receiver programs terminate after the file transfer completes. You must think carefully on how you can ensure this. We will still grade the programs that do not terminate after the file transfer for the remaining 5 marks.

   The template code given to you can be compiled and run as shown below. You are also given a test file to transfer, which can be provided as argument to the sender.

   ```
   gcc -o receiver receiver.c && ./receiver
   ```

   ```
   gcc -o sender sender.c && ./sender <filename>
   ```

   You can change the template code itself for your solution, and submit it. Please test your code thoroughly by sending different files to the receiver.